Chapter 5 Solutions

CH05-Q01

$2^{16}/2^{6} = 1024$ blocks and $1024/4 = 256$ sets give tag/index/offset widths 18/8/6.

Address	Tag	Set	Byte offset
`0x12345678`	`0x48d1`	`0x59` = 89	`0x38` = 56
`0x1a2b3c40`	`0x68ac`	`0xf1` = 241	0

Compute offset with `address & 0x3f`, set with `(address >> 6) & 0xff`, and tag with `address >> 14`.

`0x12345658` has offset 24 and the same block number as the first address. Both bases are `0x12345640`; a small numerical difference alone would not prove shared block membership near a boundary. If the block remains valid it hits.

The inclusive block range is `0x12345640` through `0x1234567f`, in set 89. Choose an invalid way first if one exists; otherwise apply the specified replacement rule.

CH05-Q02

Arrangement	Sets	Ways/comparators per selected set	Tag/index/offset	Total tag bits
Direct	1024	1	16/10/6	16384
Two-way	512	2	17/9/6	17408
Four-way	256	4	18/8/6	18432
Fully associative	1	1024	26/0/6	26624

All have 1024 lines. Total tag storage is line count times tag width. Higher associativity adds comparisons, match wiring, and way selection. A lower conflict rate may fail to offset the additional hit delay in $AMAT = H + mP$. Energy and area can also constrain the choice.

CH05-Q03

Model A gives $1 + 0.06(80) = 5.8$ cycles/access.

Model B's instruction contribution is $0.02(80) = 1.6$ cycles/instruction. Data contributes $0.30(0.08)(80) = 1.92$. Thus

Chapter 5 Solutions (continued)

$$CPI = 1.30 + 1.60 + 1.92 = 4.82, \quad T = \frac{2 \times 10^8 (4.82)}{2.5 \times 10^9} = 0.3856 \text{ s}.$$

Out-of-order work can execute during a miss, and multiple misses can overlap. The full-stall sum overestimates waiting when independent instructions, window capacity, and memory-level parallelism permit hiding. Contention can introduce other costs not represented here.

CH05-Q04

$$T_A = 1 + 4(15) + 4 = 65, \quad T_B = 1 + 2(15) + 2 = 33, \quad T_C = 1 + 2(15) + 4 = 35.$$

Effective payload rates are $16/65 \approx 0.246$, $16/33 \approx 0.485$, and $16/35 \approx 0.457$ bytes/bus cycle.

C combines two-word activations but still transfers one word each bus cycle. At activation cost 40, times become 165, 83, and 85 cycles. B's speedup over A is $165/83 \approx 1.988$; C's is $165/85 \approx 1.941$. Both gain mainly by reducing activations. The difference between them remains the two extra transfer cycles.

CH05-Q05

Global misses are $.04(.25) = .01$ per original reference. Every L1 miss pays the L2 lookup; only a quarter of those also pay memory.

$$AMAT = 1 + .04(8) + .04(.25)(100) = 2.32 \text{ cycles}.$$

Using .25 as global gives $1 + .04(8) + .25(100) = 26.32$, incorrectly applying memory cost to far too many original references. L2 serves 75% of L1 misses. Halving L1 misses gives global L2 miss rate $.02(.25) = .005$.

CH05-Q06

Page offset is 12 bits. VPN is `0x00403`; offset is `0xabc` or 2748. A single-level table contains 2^{20} entries, so size is $2^{20}(4) = 2^{22}$ bytes, four MiB.

The first index is `(VA >> 22) & 0x3ff = 1`. The second is `(VA >> 12) & 0x3ff = 3`.

$$PA = (0x5f \ll 12) \mid 0xabc = 0x5fabc.$$

A denied store is a protection fault despite the page being present. The OS may terminate, signal, or perform a legitimate repair such as a defined copy-on-write operation. It cannot silently complete an unauthorized write.

Chapter 5 Solutions (continued)

CH05-Q07

Preserve W's dirty content in backing storage or an independently safe pending-writeback buffer before reusing F. Update ownership and mappings so no valid stale translation points into newly assigned content. Only then install V and authorize the retried access.

An early store could leave a partial or unauthorized effect if recovery fails. Precise state retains enough address, data, and access information to retry without losing or duplicating effects.

Changing a PTE does not automatically remove an old cached translation on every processor. Invalidate relevant TLB entries or use a mechanism that makes stale generations unusable. ASID tags distinguish same-VPN entries across address spaces, avoiding unconditional full flushes on every switch. They do not prove read/write permission or replace coherence.

CH05-Q08

```
cycle1: Idle,read request -> Compare Tag
cycle2: clean-victim miss -> Allocate
cycle3: Ready=0 -> Allocate
cycle4: Ready=0 -> Allocate
cycle5: Ready=1,install block -> Compare Tag
cycle6: valid/tag hit,return selected word,CacheReady -> Idle
```

This is three Allocate cycles, not three waiting cycles plus a fourth Ready cycle.

```
Idle,store request
-> Compare Tag,dirty-victim miss
-> Write-Back,Ready=1
-> Allocate,Ready=1
-> Compare Tag,hit: apply saved store,set dirty,CacheReady
-> Idle
```

Latch address, request type, and store data on the Idle-to-Compare-Tag acceptance edge and retain them until the transaction completes, using a request register, queue, or dedicated state. Do not wait for a miss and then rely on processor inputs that may have changed. Otherwise the filled block cannot be associated with its required read or write.

Allocation overwrites the victim's data/tag/dirty state. Preserve dirty contents first in memory or a buffer with guaranteed completion. Losing the only newest copy destroys committed architectural memory and recovery correctness, not merely performance.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Chapter 5 Solutions (continued)

CH05-Q09

Event	P1	P2	Memory
Initial	I	I	Old value current
P1 read miss	S(old)	I	Old value current
P2 read miss	S(old)	S(old)	Old value current
P1 write X=1	M(1)	I	Old value may be stale
P2 actually reads X, BusRd	S(1)	S(1)	Updated to 1

P1 obtains exclusivity and invalidates P2's valid S(old) before writing X=1. Only once that other readable copy is invalid can P1 hold M(1). The final read's supplier is the former M owner P1, not stale memory. P1 supplies 1 and changes M(1) to S(1); P2 changes I to S(1), and this basic write-back MSI transaction updates memory to 1. A dirty shared owner with stale memory requires an Owned state in an extended protocol.

M means this cache holds the only valid cached copy. Allowing another valid S while M exists could leave two readable copies with different values during a write or before write-back. It violates M's exclusivity and S's current-memory condition.

Without common same-location write serialization, processors could disagree about the order of writes and retain incompatible observations. Coherence supplies that per-location contract. Consistency constrains cross-location ordering and synchronization. Neither alone makes a racing application deterministic.

CH05-Q10

In A, block numbers are `0x100`, `0x102`, `0x100`, all mapping to index zero. The first two accesses are compulsory. The last misses because of conflict; a two-line fully associative LRU cache would still hold both distinct blocks.

In B, the first three distinct blocks cause compulsory misses. LRU evicts `0x1000` on the third access. Its final revisit therefore causes a capacity miss even in the fully associative baseline. A different replacement rule might retain it, which is why the question specifies LRU.

Two-way A has one set with two lines. Its sequence is miss, miss, hit. More ways or a victim cache can reduce conflicts, with additional tags, multiplexers, energy, or lookup delay. The net benefit depends on the full AMAT and workload.

Chapter 5 Solutions (continued)

CH05-Q11

An asterisk denotes dirty. Physical ways do not move on LRU updates.

Step	Access	Way 0	Way 1	MRU/LRU	Lower action
0	initial	empty	empty	none	none
1	R A	A	empty	A/none	read A
2	R B	A	B	B/A	read B
3	W A	A*	B	A/B	none
4	W C	A*	C*	C/A	evict clean B, read C
5	W D	D*	C*	D/C	write dirty A, then read D

Four block reads plus one write-back make five 64-byte transfers, 320 bytes. Way 0 replaces A with D while C remains in way 1. W(A) is a hit with no immediate lower update.

Under WT/NA, reads A/B fetch two blocks. W(A) hits and writes through; W(C)/W(D) write through without allocation. A/B remain. Traffic is $2(64) + 3(4) = 140$ bytes. Five actions need not be equal-size blocks. Final WB dirty C/D would add 128 bytes if a flush were required, but it is excluded.

Index, valid, and tag determine hit. Dirty records whether the lower copy is current. Overwriting dirty A before preserving it discards the only newest copy while memory remains stale, losing a committed store.

CH05-Q12

$$VPN = 0x0badf, \qquad pageOffset = 0x00d.$$

There are $32768/32 = 1024$ lines and 512 sets. Block offset=5 bits, index=9 bits, and PIPT tag=18 bits. Virtual index is `(VA>>5)&0x1ff`; block offset is `VA&0x1f`.

Index bits 13:5 plus offset require 14 bits>12. Aliases VA 0 and `0x1000` select sets 0 and 128. Comparing only PA[31:14] omits physical bits 13:12 and permits false hits. Full 20-bit PPN comparison, or equivalent identification, is needed, but still does not remove cross-set synonyms. Capacity/ways$\le$4 KiB requires at least 8 ways; 8 ways gives 128 sets and 5+7=12. No unstated coloring assumption repairs the original configuration.

$$L1index = (0x0badf \gg 10)\ \&\ 0x3ff = 0x2e, \qquad L2index = 0x0badf\ \&\ 0x3ff = 0x2df.$$

$$L1PTE = 0x00800000 + 0x2e(4) = 0x008000b8.$$

Chapter 5 Solutions (continued)

L1 PPN `0x000031` gives table base `0x00031000`:

$$L2PTE = 0x00031000 + 0x2df(4) = 0x00031b7c.$$

$$PA = (0x0001a \ll 12) \mid 0x00d = 0x1a00d.$$

Read completes if read permissions allow it. Store faults because write 0; it must not modify cache or memory first.

$$T_{miss} = 2(25) + 10 + 3 = 63 \text{ cycles}, \qquad T_{miss}/T_{hit} = 63/3 = 21.$$

This includes only the declared table reads, management, and data lookup. Page faults, disk I/O, additional fills, and contention require separate terms.

CH05-Q13

DM: 4 offset, 2 index, 26 tag bits; post-access states:

#	block	DM [line 0,1,2,3]	DM	FA LRU → MRU	FA	DM class
1	0	`[0,–,–,–]`	M	`[0]`	M	compulsory
2	4	`[4,–,–,–]`	M	`[0,4]`	M	compulsory
3	0	`[0,–,–,–]`	M	`[4,0]`	H	conflict
4	1	`[0,1,–,–]`	M	`[4,0,1]`	M	compulsory
5	2	`[0,1,2,–]`	M	`[4,0,1,2]`	M	compulsory
6	3	`[0,1,2,3]`	M	`[0,1,2,3]`	M	compulsory
7	4	`[4,1,2,3]`	M	`[1,2,3,4]`	M	capacity
8	0	`[0,1,2,3]`	M	`[2,3,4,0]`	M	capacity
9	1	`[0,1,2,3]`	H	`[3,4,0,1]`	M	hit

DM has eight misses: five compulsory, one conflict, and two capacity, plus one hit. FA also has eight misses, with its only hit at access 3. At access 3, DM misses and FA hits, contributing +1 to their difference. At access 9, DM hits and FA misses, contributing −1. A zero total difference does not erase access 3's conflict. Access 9 is not an actual miss and must not be counted as an actual capacity miss.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Chapter 5 Solutions (continued)

Independent per-set LRU in the two-way design gives:

#	block	set 0: LRU → MRU	set 1: LRU → MRU	H/M
1	0	`[0]`	`[]`	M
2	4	`[0,4]`	`[]`	M
3	0	`[4,0]`	`[]`	H
4	1	`[4,0]`	`[1]`	M
5	2	`[0,2]`	`[1]`	M
6	3	`[0,2]`	`[1,3]`	M
7	4	`[2,4]`	`[1,3]`	M
8	0	`[4,0]`	`[1,3]`	M
9	1	`[4,0]`	`[3,1]`	H

Accesses 3 and 9 hit, giving seven misses. Access 7 replaces LRU block 0 in set 0 with 4; access 8 replaces 2. Neither touches set 1, which retains blocks 1 and 3.

Total time is `references×hit time + misses×additional penalty`: $9+8P$ for DM and $11.25+7P$ for two-way. At P=20 they take 169 and 151.25 cycles. Two-way is faster when $11.25+7P<9+8P$, hence $P>2.25$. They tie at P=2.25; DM is faster for $0 \leq P<2.25$. FA's hit time is unspecified, so equal miss counts do not imply equal execution times.

Chapter 5 Solutions (continued)

Verification code appendix: cache_address_lab.py

```
"""Verifiable teaching examples for cache decoding and page translation."""

from dataclasses import dataclass


@dataclass(frozen=True)
class AddressFields:
    tag: int
    set_index: int
    offset: int


@dataclass(frozen=True)
class CacheSpec:
    capacity_bytes: int = 64 * 1024
    block_bytes: int = 64
    ways: int = 4
    address_bits: int = 32

    @property
    def blocks(self) -> int:
        return self.capacity_bytes // self.block_bytes

    @property
    def sets(self) -> int:
        return self.blocks // self.ways

    @property
    def offset_bits(self) -> int:
        return self.block_bytes.bit_length() - 1

    @property
    def index_bits(self) -> int:
        return self.sets.bit_length() - 1

    @property
    def tag_bits(self) -> int:
        return self.address_bits - self.offset_bits - self.index_bits

    def decode(self, address: int) -> AddressFields:
        offset_mask = self.block_bytes - 1
        index_mask = self.sets - 1
        return AddressFields(
            tag=address >> (self.offset_bits + self.index_bits),
            set_index=(address >> self.offset_bits) & index_mask,
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Chapter 5 Solutions (continued)

```
            offset=address & offset_mask,
        )


class SetAssociativeCache:
    def __init__(self, spec: CacheSpec) -> None:
        self.spec = spec
        self.entries: list[list[tuple[int, int]]] = [[] for _ in range(spec.sets)]
        self.clock = 0

    def access(self, address: int) -> bool:
        self.clock += 1
        fields = self.spec.decode(address)
        ways = self.entries[fields.set_index]
        for position, (tag, _) in enumerate(ways):
            if tag == fields.tag:
                ways[position] = (tag, self.clock)
                return True
        if len(ways) < self.spec.ways:
            ways.append((fields.tag, self.clock))
        else:
            victim = min(range(len(ways)), key=lambda i: ways[i][1])
            ways[victim] = (fields.tag, self.clock)
        return False


@dataclass
class CacheLine:
    tag: str
    last_used: int
    dirty: bool = False


def write_back_trace(operations: list[tuple[str, str]], ways: int = 2) -> dict:
    lines: list[CacheLine] = []
    read_blocks: list[str] = []
    write_backs: list[str] = []
    snapshots: list[dict] = []
    clock = 0
    for operation, tag in operations:
        clock += 1
        matched = next((line for line in lines if line.tag == tag), None)
        if matched is not None:
            matched.last_used = clock
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter 5 Solutions (continued)

```
            if operation == "W":
                matched.dirty = True
            snapshots.append({"event": f"{operation} {tag}", "result": "hit", "lines":
[line.tag for line in lines]})
            continue

        if len(lines) == ways:
            victim = min(lines, key=lambda line: line.last_used)
            if victim.dirty:
                write_backs.append(victim.tag)
            lines.remove(victim)
        lines.append(CacheLine(tag=tag, last_used=clock, dirty=operation == "W"))
        read_blocks.append(tag)
        snapshots.append({"event": f"{operation} {tag}", "result": "miss", "lines": [line.tag
for line in lines]})
    return {
        "snapshots": snapshots,
        "read_blocks": read_blocks,
        "write_backs": write_backs,
        "set": [{"tag": line.tag, "dirty": line.dirty} for line in sorted(lines, key=lambda
line: line.last_used)],
    }


def translate_two_level(va: int, ppn: int | None, page_bits: int = 12) -> tuple[int, int,
int, int, int | None]:
    vpn = va >> page_bits
    offset = va & ((1 << page_bits) - 1)
    level1 = (vpn >> 10) & 0x3FF
    level2 = vpn & 0x3FF
    pa = None if ppn is None else (ppn << page_bits) | offset
    return vpn, offset, level1, level2, pa


def main() -> None:
    spec = CacheSpec()
    assert (spec.blocks, spec.sets) == (1024, 256)
    assert (spec.tag_bits, spec.index_bits, spec.offset_bits) == (18, 8, 6)

    first = spec.decode(0x12345678)
    second = spec.decode(0x1A2B3C40)
    same_block = spec.decode(0x12345658)
    assert first == AddressFields(0x48D1, 0x59, 0x38)
    assert second == AddressFields(0x68AC, 0xF1, 0)
    assert (same_block.tag, same_block.set_index, same_block.offset) == (0x48D1, 0x59, 0x18)

    cache = SetAssociativeCache(spec)
    addresses = [0x12345678, 0x1A2B3C40, 0x2BADF00C, 0x40001020, 0x12345658]
    results = [cache.access(address) for address in addresses]
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Chapter 5 Solutions (continued)

```
    assert results == [False, False, False, False, True]

    vpn, offset, level1, level2, pa = translate_two_level(0x00403ABC, 0x0005F)
    assert (vpn, offset, level1, level2, pa) == (0x403, 0xABC, 1, 3, 0x5FABC)
    assert translate_two_level(0x00403ABC, None)[-1] is None

    write_trace = write_back_trace([("R", "A"), ("R", "B"), ("W", "A"), ("W", "C"), ("W",
"D")])
    assert write_trace["read_blocks"] == ["A", "B", "C", "D"]
    assert write_trace["write_backs"] == ["A"]

    vipa = 0x0BADF00D
    ptw_vpn = vipa >> 12
    l1_index = (ptw_vpn >> 10) & 0x3FF
    l2_index = ptw_vpn & 0x3FF
    l1_pte_address = 0x00800000 + l1_index * 4
    l2_pte_address = (0x000031 << 12) + l2_index * 4
    final_pa = (0x00001A << 12) | (vipa & 0xFFF)
    assert (ptw_vpn, l1_index, l2_index) == (0x0BADF, 0x2E, 0x2DF)
    assert l1_pte_address == 0x008000B8
    assert l2_pte_address == 0x00031B7C
    assert final_pa == 0x1A00D
    assert 2 * 25 + 10 + 3 == 63

    amat = 1 + 0.04 * 8 + 0.04 * 0.25 * 100
    assert abs(amat - 2.32) < 1e-12

    print("cache: 64 KiB, 4-way, 64-byte blocks")
    print(f"blocks={spec.blocks}, sets={spec.sets}, tag/index/offset=
{spec.tag_bits}/{spec.index_bits}/{spec.offset_bits}")
    for address, result in zip(addresses, results):
        fields = spec.decode(address)
        status = "HIT" if result else "MISS"
        print(f"0x{address:08x}: tag=0x{fields.tag:x}, index=0x{fields.set_index:x}, offset=
{fields.offset:2d}: {status}")
    print(f"translation: VPN=0x{vpn:05x}, L1={level1}, L2={level2}, PA=0x{pa:05x}")
    print(f"AMAT model: {amat:.2f} cycles")
    print(f"write-back trace: reads={write_trace['read_blocks']}, write-backs=
{write_trace['write_backs']}")
    print(f"page walk: L1 PTE=0x{l1_pte_address:08x}, L2 PTE=0x{l2_pte_address:08x},
PA=0x{final_pa:x}")
    print("cache address lab: PASS")


if __name__ == "__main__":
    main()
```

Chapter 5 Solutions (continued)

References and attribution

David A. Patterson, John L. Hennessy. Computer Organization and Design: The Hardware/Software Interface, Revised Fourth Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study